

Practical Implementation and Evaluation of Ethereum’s Proof-of-Stake Bouncing Attack and Patch on MOBS

Ricardo Loureiro^{1,3}, António Ravara^{1,3} and Simão Melo de Sousa^{2,3}

¹DI, FCT, NOVA

² DEEL, FCT, University of Algarve

³NOVA-LINCS

Abstract

With Ethereum going through *The Merge* and moving from a Proof-of-Work to a Proof-of-Work consensus it has opened itself to a new range of attacks. One of this attacks was a bouncing attack on liveness that prevented the blockchain from finalizing new blocks, the Ethereum community then responded with a patch to fix this vulnerability. An article analyzing this vulnerability and subsequent patch has found that there is a new vulnerability on the patched protocol that results in the protocol only having probabilistic liveness. The article only provides a theoretical analysis and pseudo-code which motivated us to implement Ethereum as described, the attack and subsequent patch in MOBS. We therefore provide empirical evidence to complement the theoretical analysis supporting the claims made on the article.

1 Introduction

Consensus and blockchain protocols are usually described with pseudocode and model checked with idealized languages that not reflect the implementations. Since validating correctness is very costly, before planning an implementation, developers should test their ideas and prototypes. There is a lack of methodologies to develop, evaluate, tune and replicate these protocols, opening a need for extensible and modular tools for consensus analysis and experimental labs for rapid prototyping. With this in mind MOBS was developed with the aim to give the ability to test and validate these protocols under different conditions and settings by changing the execution parameters, aiming to catch vulnerabilities or even logic errors before deploying changes to these protocols. Since verification of protocols is very costly, developers can use MOBS to get confidence in their implementations from experimentations first.

Consensus protocols are inherently complex to design and implement correctly. In blockchain systems, where financial transactions and dynamic behavior are the primary features, protocol correctness is essential, as even minor errors can lead to substantial financial losses and system malfunctions. One illustration of this is the transition of Ethereum from a Proof of Work to a Proof of Stake consensus. This transition exposed the protocol to vulnerabilities that could be exploited by bouncing attacks on liveness. The chain is unable to be finalized as a result of this type of attack, as the primary selected chain in the fork choice rule is perpetually hopping between two alternative branches.

Liveness in a blockchain protocol is defined as the ability to always finalize new blocks. These blocks in the Ethereum protocol are finalized through a process called Casper FFG (Friendly Finality Gadget) [1], in combination with the LMD GHOST (Latest Message Driven Greediest Heaviest Observed SubTree) fork choice rule [2].

The Bouncing Attack is an attack that prevents the chain from being finalized by making the main chain selected in the fork choice rule continually bounce between two candidate chains. The attack leverages the fact that candidate chains should start from the justified checkpoint

with the highest epoch, and Byzantine nodes exploit this by dividing honest validators opinions, justifying a new checkpoint after some honest validators already have cast their vote.

In [3] this attack and with the Ethereum protocol are specified and analyzed along with the patch implemented by the Ethereum community. In this article is also stated and formalized that the patch only provides probabilistic liveness but no implementation is given. Providing an implementation in MOBS complements the article by producing an executable, reproducible reference: it validates the paper’s assumptions, uncovers implementation-level corner cases, and allows empirical measurement of attack success rates and the patch’s effectiveness under realistic network and Byzantine conditions. This practical reproduction strengthens the article’s claims by enabling repeatable experiments, quantitative evidence, and clearer guidance for protocol implementers.

This document serves as a complement and provides more detailed information about our implementation and how it relates to the sdescription given in the article.

Here we show and detail our implementation in MOBS along with the re-creation of the Bouncing Attack, the patch proposed by the Ethereum community and how this patch only provides probabilistic liveness.

2 Implementation

Our implementation was done in MOBS a simulator makes use of OCaml’s *modules* and *functors* to provide modularity and extensibility. MOBS adopts a *Discrete-Event Simulation Model* making the state of the system only change in discrete points in time when events occur. These events are stored in a queue ordered with two main values:

- **Timestamp:** This value dictates the order in which the events are stored in the queue and is based on the simulator’s internal clock. When getting a new event the simulator will fetch from the queue the one with the smallest timestamp and move its internal clock to match that event’s timestamp.
- **Target:** The entity that should process this event.

Events are fetched from the queue until no more events remain or a predefined stopping condition is reached.

For our implementation in MOBS we followed the description provided in pseudocode provided in [3]. We focused on the parts pertaining to the consensus mechanism and to allow a faithful replication of the results ignored some of the randomness mechanisms that were described like choosing the member of the Proposers and the Validators committee. We also assumed that each attestation has a weight of 1 when the protocol was running for simplicity.

The first algorithm presented in the article 1 represents the main execution loop. In our implementation 2 we continuously every node sends a message to itself at the end of the main loop execution to execute the main code again. Also instead of extracting the logic of *prepareBlock()* and *prepareAttestation()* to their own methods we keep them explicit in the main code.

■ **Algorithm 1** Main code for a validator p

```

1: procedure VALIDATORMAIN( )
2:    $tree_p := nil$  ▷ The tree represents the linked received blocks
3:    $role_p := []$  ▷  $role_p$  can be ROLE_PROPOSER and/or ROLE_ATTESTER when it
   is not empty
4:    $slot_p := 0$  ▷  $slot_p \in \mathbb{N}$ 
5:    $lastJustifiedCheckpoint_p := (0, genesisBlock)$  ▷ A checkpoint is a tuple (epoch,
   block)
6:    $attestation_p := []$  ▷ List of latest attestations received for each validator
7:    $validatorIndex_p := \text{index of the validator}$  ▷ Each validator has a unique index
8:    $listValidator := [p_0, p_1, \dots, p_{N-1}]$  ▷ A list of the validators index
9:    $balances := []$  ▷ A list of the balances of the validators, their stake
10:
11:   start  $\text{sync}(tree_p, slot_p, attestation_p, lastJustifiedCheckpoint_p, role_p, balances)$ 
12:
13:   while true do
14:     if  $role_p \neq \emptyset$  then
15:       if  $ROLE\_PROPOSER \in role_p$  then
16:          $\text{prepareBlock}()$ 
17:       if  $ROLE\_ATTESTER \in role_p$  then
18:          $\text{prepareAttestation}()$ 
19:        $role_p \leftarrow []$ 
20:     else
21:       no role assigned ▷ No action required

```

Figure 1: Article algorithm 1

```

t->a->b->t
let receive_main (node:t) _ _ =
  if (node.data.proposer && (node.data.slot == node.id || node.id == 3)) then
  begin (* prepareBlock() logic*)
    (* if byzantine && node == 3 then do nothing until slot 30 then send propose with slot = 0 *)
    let path : block list = find_deepest_path node.data.tree in
    let parent =
      match List.find_opt (fun (b : block) -> b.epoch = node.data.epoch - 1) path with
      | Some p -> p
      | None -> get_block_from_tree node.data.tree
    in
    let parent_hash = parent.hash in
    let new_node : block = { hash = Printf.sprintf "%x%x%x" (Random.bits ()) (Random.bits ()) (Random.bits ());
                             epoch = node.data.epoch; slot = node.data.slot;
                             content = parent_hash; justified = false;
                             finalized = false } in
    node.data.tree <- insert_block node.data.tree ~block:new_node ~parent_hash_opt:(Some parent_hash);
    if (Constants.byzantine_execution && (node.id == 3) && (node.data.slot > 28) && node.data.epoch > 100) then
    begin
      node.data.proposer <- false;
      EthereumNetwork.gossip node.id (Propose(node.id, node.data.epoch, node.data.slot, node.data.tree, new_node));
    end
    else if (node.id <> 3) then
    begin
      node.data.proposer <- false;
      EthereumNetwork.gossip node.id (Propose(node.id, node.data.epoch, node.data.slot, node.data.tree, new_node));
    end
  end;

  if (node.data.slot == 32 && node.id == 1) then EthereumNetwork.send node.id node.id (JustificationFinalization(node.id));

  if (node.data.attestations_sent < 3 && node.data.slot >= 11) then
  begin (* prepareAttestation() logic*)
    node.data.attestations_sent <- node.data.attestations_sent + 1;
    let root_block = get_block_from_tree node.data.tree in
    let last_justified_checkpoint =
      match deepest_justified_on_path node.data.tree with
      | Some b -> Leaf b
      | None -> Leaf root_block in
    let current_checkpoint =
      match get_latest_checkpoint node.data.tree with
      | Some b -> Leaf b
      | None -> Leaf root_block in
    EthereumNetwork.gossip node.id (Attestation(node.id, node.data.epoch, current_checkpoint, last_justified_checkpoint));
  end;
node

```

Figure 2: MOBS `receive_main` function

In our implementation of `broadcastBlock()` 3 we kept the same flow of the presented algorithm with the only branches of execution being for the byzantine execution.

Algorithm 2 broadcast block

-
- 1: **procedure** PREPAREBLOCK()
 - 2: $headBlock_p := \text{getHeadBlock}()$
 - 3: $randaoReveal := \text{sign}(slot, epochOf(slot))$
 - 4: **broadcast** $\langle PROPOSE, (slot, hash(headBlock_p), randaoReveal, content) \rangle$
-

Figure 3: Article algorithm 2

For `broadcastAttestation()` 4 we ignored the Checkpoint vote logic since there was no description on how a node should vote for the checkpoint and how it should use their stake on the vote. We did the assumption in our execution that every node always voted with the same stake.

Algorithm 3 Broadcast Attestation

```

1: procedure PREPAREATTESTATION( )
2:   waitForBlockOrOneThird()  $\triangleright$  wait for a new block in this slot or  $\frac{1}{3}$  of the slot
3:    $headBlock_p := \text{getHeadBlock}()$ 
4:    $currentCheckpoint_p := (\text{first block of the epoch}, \text{epochOf}(slot))$ 
5:    $CheckpointVote_p := (\text{lastJustifiedCheckpoint}_p, \text{currentCheckpoint}_p)$ 
6:   broadcast  $\langle ATTEST, (slot_p, \underbrace{\text{hash}(headBlock_p)}_{\text{block vote}}, \underbrace{CheckpointVote_p}_{\text{checkpoint vote}}) \rangle$ 

```

Figure 4: Article algorithm 3

For *sync()* 5 we did not explicitly implement this method. Since we determined the attestation and proposer quorums to be static. The logic to call the *justificationFinalization* is done in the *receive_main* method between the *prepareBlock()* and *prepareAttestation()* methods.

Algorithm 4 Sync

```

1: procedure SYNC(tree, slot, attestation, role, lastJustifiedCheckpoint,)
2:   start  $\text{syncBlock}(slot, tree)$ 
3:   start  $\text{syncAttestation}(attestation)$ 
4:   repeat
5:      $previousSlot \leftarrow slot$ 
6:      $slot \leftarrow \lfloor \text{time in seconds since genesis block} / 12 \rfloor$ 
7:     if  $previousSlot \neq slot$  then  $\triangleright$  If we start a new slot
8:        $roleSlotDone \leftarrow \text{false}$ 
9:       if  $\text{computeProposerIndex}(\text{getSeed}(\text{current epoch})) = \text{validatorIndex}$  then
10:         $role_p \leftarrow \text{ROLE\_PROPOSER}$ 
11:      if  $slot \pmod{32} = 0$  then  $\triangleright$  First slot of an epoch
12:         $\text{justificationFinalization}(tree, \text{lastJustifiedCheckpoint})$ 
13:         $\text{getCommitteeAssignment}(\text{next epoch}, \text{validatorIndex}, \text{listValidator})$ 
14:   until validator exit

```

Figure 5: Article algorithm 4

For *syncBlock()* 5 and *syncAttestation()* 7 we skipped over the *isValid()* method since there was no description on the implementation of this method, and we assumed that the byzantine would not be interfering with these messages. Our implementation of *syncBlock()* 8 just updates the node's tree with the new block. To simplify the process and avoid concurrency races we just update the whole tree with the received tree. For our implementation of *syncAttestation()* we follow the same logic and save the information of the attestation in a hash table where each key is the epoch and the value is a list of attestation received. This will be used later for the justification and finalization logic.

■ **Algorithm 5** Sync Block

```
1: procedure SYNCBLOCK(slot, tree)
2:   upon  $\langle PROPOSE, (slot_i, hash(headBlock_i), randaoReveal_i, content_i) \rangle$  from
    validator i do
3:     block :=  $\langle PROPOSE, (slot_i, hash(headBlock_i), randaoReveal_i, content_i) \rangle$ 
4:     if isValid(block) then
5:       add block to tree
6:       if slot (mod 32) ≤ 8 then
7:         update justified checkpoint if necessary
```

Figure 6: Article algorithm 5

■ **Algorithm 6** Sync Attestation

```
1: procedure SYNCATTESTATION(attestation)
2:   upon  $\langle ATTEST, (slot_i, headBlock_i, checkpointEdge_i) \rangle$  from validator i do
3:     attestationi :=  $\langle ATTEST, (slot_i, headBlock_i, checkpointEdge_i) \rangle$ 
4:     if isValid(attestation) then
5:       attestation[i] ← attestationi
```

Figure 7: Article algorithm 6

```
t -> 'a -> 'b -> 'c -> block ethereum_tree -> 'd -> t
let receive_propose (node:t) _ _ _ tree _ =
  (* PATCH *)
  (* if (node.data.slot < 24) then *)
  node.data.tree <- tree;
  node
```

Figure 8: MOBS receive_propose

```

t -> int -> int -> block ethereum_tree -> block ethereum_tree -> t
let receive_attestation (node:t) sender epoch block checkpoint =
  let current_list =
    match Hashtbl.find_opt node.data.attestation_quorum epoch with
    | Some l -> l
    | None -> []
  in
  let filtered_list = List.filter (fun (s, _, _) -> s <> sender) current_list in
  let new_list = (sender, block, checkpoint) :: filtered_list in
  Hashtbl.replace node.data.attestation_quorum epoch new_list;
  node

```

Figure 9: MOBS receive_attestation

For *getHeadBlock()* 10 we implemented the logic for this under the helper methods to navigate and operate the n-ary tree that represents the blockchain. I would suggest consulting the repository and checking the *deepest_justified_on_path* method since it would not be feasible to show the code through images because of the other helper methods that *deepest_justified_on_path* makes use of.

■ Algorithm 7 Get Head Block

```

1: procedure GETHEADBLOCK( )
2:   block ← block of the justified checkpoint with the highest epoch
3:   while block has at least one child do
4:     block ←  $\arg \max_{b' \text{ child of } block} \text{weight}(tree, Attestation, b')$ 
5:     (ties are broken by hash of the block header)
6:   return block

```

Figure 10: Article algorithm 7

The *getWeight()* 11 was not implemented due to the fact that we assumed every node was voting with a stake of 1.

■ Algorithm 8 Weight

```

1: procedure WEIGHT(tree, Attestation, block)
2:   w ← 0
3:   for every validator vi do
4:     if  $\exists a \in Attestation$  an attestation of vi for block or a descendant of block then
5:       w ← w + stake of vi
6:   return w

```

Figure 11: Article algorithm 8

Algorithm 9 12 defines the consensus logic for the protocol. To implement this we followed the same logic as presented in the article like shown in 13. When any node is finalized we propagate a *FinalizedNode* message, this acts as a logging mechanism and as away to converge every node to the same tree.

■ **Algorithm 9** Justification and Finalization

```

1: procedure JUSTIFICATIONFINALIZATION(tree, lastJustifiedCheckpoint)
2:   source  $\leftarrow$  lastJustifiedCheckpoint
3:   target  $\leftarrow$  the current checkpoint
4:   nbCheckpointVote  $\leftarrow$  countMatchingCheckpointVote(source, target)
5:    $\triangleright$  justification process:
6:   if nbCheckpointVote  $\geq \frac{2}{3}$  * total balance of validators then
7:     target.justified = true
8:     lastJustifiedCheckpoint  $\leftarrow$  target
9:    $\triangleright$  finalization process:
10:  A, B, C, D  $\leftarrow$  the last 4 checkpoints  $\triangleright$  With D being the current checkpoint.
11:  if A.justified  $\wedge$  B.justified  $\wedge$  (A  $\xrightarrow{J}$  C) then
12:    D.finalized = true  $\triangleright$  Finalization of A
13:  if B.justified  $\wedge$  (B  $\xrightarrow{J}$  C) then
14:    B.finalized = true  $\triangleright$  Finalization of B
15:  if B.justified  $\wedge$  C.justified  $\wedge$  (B  $\xrightarrow{J}$  D) then
16:    B.finalized = true  $\triangleright$  Finalization of B
17:  if C.justified  $\wedge$  (C  $\xrightarrow{J}$  D) then
18:    C.finalized = true  $\triangleright$  Finalization of C

```

Figure 12: Article algorithm 9

```

(* For simplicity, assume a fixed number of nodes, in this case, 10 *)
t->t
let receive_justification_finalization(node:t) =
  let source_opt = deepest_justified_on_path node.data.tree in
  let target_opt = get_latest_checkpoint node.data.tree in
  (match source_opt, target_opt with
  | Some source, Some target ->
    let nb_checkpoint_vote = count_matching_checkpoint_vote node source target in
    if nb_checkpoint_vote > 7 then target.justified <- true;
    let a, b, c, d = get_last_four_checkpoints node.data.tree in
    if a.justified && b.justified && (supermajority_link node a c) then
      begin
        a.finalized <- true;
        EthereumNetwork.gossip node.id (FinalizedNode(node.id, node.data.epoch, node.data.slot, node.data.tree, a));
      end
    else if b.justified && (supermajority_link node b c) then
      begin
        b.finalized <- true;
        EthereumNetwork.gossip node.id (FinalizedNode(node.id, node.data.epoch, node.data.slot, node.data.tree, b));
      end
    else if b.justified && c.justified && (supermajority_link node b d) then
      begin
        b.finalized <- true;
        EthereumNetwork.gossip node.id (FinalizedNode(node.id, node.data.epoch, node.data.slot, node.data.tree, b));
      end
    else if c.justified && (supermajority_link node c d) then
      begin
        c.finalized <- true;
        EthereumNetwork.gossip node.id (FinalizedNode(node.id, node.data.epoch, node.data.slot, node.data.tree, c));
      end
    | _ -> assert(false)
  );
  node

```

Figure 13: MOBS receive_justification_finalization

3 Results

Our implementation allowed us to recreate the scenarios presented in [3] and validate the claims made in the article. The first scenario we tested was a basic execution of the protocol without any Byzantine nodes to validate that the protocol was able to finalize new blocks as expected. To validate this execution we ran the simulation for a predefined amount of time and checked that new blocks were being finalized. To help with this validation we logged every time a node finalized a new block and at the end of the simulation we used a python script to parse the logs and check that a new block was finalized for each epoch.

```
Epoch 119: Finalized block hash: c54c41e2c141f8c6d429
Epoch 120: Finalized block hash: 5a09eab2e223e3717ed2057
Epoch 121: Finalized block hash: 362b5bba136236a72dc37ae
Epoch 122: Finalized block hash: 3daa9e6026ed85f46fa288a
Epoch 123: Finalized block hash: a803a06ad48a8c2cf507bd
Epoch 124: Finalized block hash: 9845ee12e0131e6dbf3280
Epoch 125: No FinalizedNode messages
```

Figure 14: Output from the log analyzer script for an Ethereum correct execution

In our second scenario we implemented Byzantine behavior for some nodes in order to recreate the bouncing attack as described in the article. This behavior would start at a predefined epoch and would make the Byzantine nodes split the honest validators opinions by justifying a new checkpoint after some honest validators already have cast their vote. We ran the simulation for the same amount of time as the previous scenario and checked that no new blocks were not being finalized.

```
Epoch 121: No FinalizedNode messages
Epoch 122: No FinalizedNode messages
Epoch 123: No FinalizedNode messages

===== Epochs without Finalized Blocks (run > 5 detected) =====
102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123
```

Figure 15: Output from the log analyzer script for an Ethereum incorrect execution

After in our next scenario we implemented the patch proposed by the Ethereum community to mitigate the bouncing attack. This patch would make nodes only consider attestations up until a pre-determined slot when justifying a new checkpoint. This would give time for correct attestators to change their vote into the correct block. After implementing this patch we ran the simulation again for the same amount of time and checked that new blocks were being finalized again even after the byzantine nodes started their attack.

In our final scenario we wanted to validate the claim made in the article that the patch only provides probabilistic liveness. To validate this we changed the behaviour of the byzantine nodes to simulate network lag. This would make the byzantine nodes justify a new checkpoint after the honest validators had already cast their vote but before the pre-determined slot defined in the patch. This would make some honest validators change their vote to another chain and prevent finalization. We ran the simulation for the same amount of time and checked that for some epochs no new blocks were being finalized.

4 Conclusion

This work provided an implementation of the Ethereum Proof-of-Stake protocol as described in [3] along with the bouncing attack and the patch proposed by the Ethereum community to mitigate this attack. Through our implementation we were able to empirically validate the claims made in the article regarding the vulnerability of the protocol to the bouncing attack and how

the proposed patch only provides probabilistic liveness. Even though our implementation made some simplifications to the protocol like assuming static quorums and every attestation having a weight of 1, the core logic of the protocol was preserved allowing us to validate the claims made in the article.

This implementation can be used as a guide for the implementation of other similar consensus protocols and blockchains. We present a way to use MOBS to validate and test these protocols before deploying them in real world scenarios, allowing developers to catch vulnerabilities and logic errors early on diverse scenarios.

References

- [1] Vitalik Buterin and Virgil Griffith. “Casper the friendly finality gadget”. In: *arXiv preprint arXiv:1710.09437* (2017).
- [2] Vitalik Buterin et al. “Combining ghost and casper”. In: *arXiv preprint arXiv:2003.03052* (2020).
- [3] Ulysse Pavloff, Yackolley Amoussou-Guenou, and Sara Tucci-Piergiovanni. *Ethereum Proof-of-Stake under Scrutiny*. 2022. arXiv: 2210.16070 [cs.CR].